

AUKTAVE 2K26 - Development Report & Interview Q&A

The build process, the questions it invites, terms answered, and sample code

Nirmalya Mandal - SAP Labs Interview Prep

Reconstructed from the 150-commit development history

Contents

How to use this report	3
Part 1 - The development story	4
Phase 1 - Scaffold and the hero (early May 2026)	4
Phase 2 - Content and events	4
Phase 3 - Motion and feel	4
Phase 4 - The TypeScript refactor	4
Phase 5 - SEO, PWA, and registration	4
Part 2 - Questions and detailed answers	5
“Why did you refactor the whole thing to TypeScript?”	5
“How is the event content structured?”	5
“Framer Motion versus GSAP - why both?”	5
“What is Lenis / why not native smooth scrolling?”	5
“How did you keep heavy animations from hurting performance?”	5
“Why Next.js for this?”	5
“What is a PWA?”	5
“sessionStorage vs localStorage - why sessionStorage for the audio gate?”	5
Part 3 - The bait-terms dictionary	7
Part 4 - Sample code snippets (pen and paper)	9
The hooks to drop on purpose	11

How to use this report

AUKTAVE is your front-end craft project: a production event portal for a university fest, built for performance and clean architecture. It shows you can ship a polished, fast, well-structured web app. Structure: **Part 1** build story, **Part 2** Q&A with answers, **Part 3** bait-terms dictionary, **Part 4** sample code. Say a term, pause, let them ask.

What it is in one breath: the official portal for a university tech fest - event discovery, schedules, sponsor showcases, and registrations - built so that adding a new event is a data change, not a code change, and tuned to stay smooth on the mid-range phones most attendees carry.

Part 1 - The development story

Phase 1 - Scaffold and the hero (early May 2026)

- Set up the **Next.js** project (React 19).
- Built a **splash screen** with a progress animation and a home **splash gate** that shows it once.
- Hero section with a background video (z-index and backdrop-blur tuning) and a **custom cursor** component.

Phase 2 - Content and events

- Modelled the fest's events as structured data; built event **detail pages** and a Robotics page with nested sub-events.
- Added an **EventPoster** component, FAQs, and downloadable rulebook PDFs per event.
- Sponsor **ticker** as a responsive **marquee**; an announcement banner; schedule and contact info.

Phase 3 - Motion and feel

- Animations with **Framer Motion** and **GSAP**.
- **Lenis** for smooth scrolling across the navbar, splash, and audio gate.
- An **audio entry gate** that asks the user's sound preference and remembers it in **sessionStorage**.
- A **Leaflet** map view for the venue, with a custom cursor for the zoom controls.
- Typography with the **Bebas Neue** font; responsive tuning for small screens.

Phase 4 - The TypeScript refactor

- **Refactored the entire codebase from JavaScript to TypeScript** (a single deliberate pass).
- This turned the event content into a **typed** architecture with **nested dynamic routing**, so each event page is generated from typed data.

Phase 5 - SEO, PWA, and registration

- **SEO**: structured metadata and Open Graph tags for clean link previews.
- **PWA** capabilities so the site is installable and works with patchy data.
- Registration via **Google Forms** plus a QR code, wired into the event pages.



Part 2 - Questions and detailed answers

“Why did you refactor the whole thing to TypeScript?”

The project started in JavaScript to move fast, but as the number of events and pages grew, I wanted the safety. TypeScript adds static types, so a typo in an event’s data or a missing field is caught at build time instead of breaking a page at runtime. It also made the content architecture self-documenting - each event has a defined shape - which is exactly what let me generate pages from typed data.

“How is the event content structured?”

Events are defined as **typed data**, separate from the presentation, and each event gets its own page through **nested dynamic routing**. So adding a new event is a data change - I add a typed object - not a code change. That separation keeps the site maintainable and consistent.

“Framer Motion versus GSAP - why both?”

Framer Motion is React-first and declarative - great for component enter/exit animations and simple, state-driven motion. GSAP is a powerful imperative animation engine, better for complex, timeline-based sequences and fine control. I used Framer Motion for the everyday component animation and GSAP where I needed a precise, orchestrated sequence.

“What is Lenis / why not native smooth scrolling?”

Lenis is a smooth-scroll library that gives consistent, controllable inertia across browsers. Native CSS smooth scrolling is inconsistent between devices and hard to sync with animations. Lenis gave me one predictable scroll behaviour I could tie animations to.

“How did you keep heavy animations from hurting performance?”

Most attendees are on mid-range phones, so I scoped motion to visible sections, respected the user’s reduced-motion preference, and kept animations lazy and interruptible rather than blocking the first paint. The goal is that the site feels alive without dropping frames on a cheap device.

“Why Next.js for this?”

Server-side rendering and generated metadata are good for SEO - people find the fest by searching - and the App Router’s file-based routing made the nested event pages clean. Plus PWA support and image optimisation out of the box.

“What is a PWA?”

A Progressive Web App - a website that can be installed to the home screen and work offline, using a service worker to cache content. Useful at a live event where phone data is patchy.

“sessionStorage vs localStorage - why sessionStorage for the audio gate?”

Both store data in the browser as key-value pairs. localStorage persists across sessions; sessionStorage lasts only for the current tab session. I used sessionStorage for the audio preference so it applies for the

visit but does not linger forever - a fresh visit asks again.

Part 3 - The bait-terms dictionary

Next.js / App Router / dynamic routing

A React framework with server rendering and file-based routing; dynamic routes generate pages from data. *Say it:* “Each event page is a dynamic route generated from typed event data.”

TypeScript refactor

Migrating JavaScript to statically typed TypeScript. *Say it:* “I refactored the whole app to TypeScript so bad event data is caught at build time.”

Framer Motion

React-first declarative animation library. *Say it:* “Framer Motion for component enter/exit motion.”

GSAP

A powerful imperative, timeline-based animation engine. *Say it:* “GSAP for the precise, orchestrated sequences.”

Lenis / smooth scrolling

A smooth-scroll library with consistent inertia. *Say it:* “Lenis gives one predictable scroll I can sync animations to.”

Custom cursor

A JavaScript-driven cursor replacing the default pointer. *Say it:* “A custom cursor that reacts to interactive elements.”

sessionStorage vs localStorage

Per-tab-session vs persistent browser storage. *Say it:* “The audio choice sits in sessionStorage, so it’s per-visit.”

Leaflet

An open-source 2D map library. *Say it:* “The venue map is Leaflet.”

PWA / service worker

An installable, offline-capable web app. *Say it:* “PWA support so it installs and survives patchy data at the event.”

SEO / Open Graph

Structured metadata and social link-preview tags. *Say it:* “Open Graph tags and metadata for search and clean shares.”

Marquee

A continuously scrolling ticker. *Say it:* “The sponsor ticker is a responsive marquee.”

Reduced-motion

Respecting the OS ‘reduce motion’ accessibility setting. *Say it:* “Motion respects reduced-motion, so it’s accessible.”

Part 4 - Sample code snippets (pen and paper)

Typed event data + a nested dynamic route (Next.js)

```
// data/events.ts
export type EventItem = {
  slug: string;
  title: string;
  prize: string;
  date: string;
};
export const events: EventItem[] = [
  { slug: "hackathon", title: "Hackathon", prize: "...", date: "..." },
];

// app/events/[slug]/page.tsx
export function generateStaticParams() {
  return events.map((e) => ({ slug: e.slug })); // one page per event
}
export default function EventPage({ params }: { params: { slug: string } }) {
  const event = events.find((e) => e.slug === params.slug);
  if (!event) return notFound();
  return <EventDetail event={event} />;
}
```

Say aloud: “Adding an event is a data change - the route generates a page per typed event object.”

Framer Motion - animate a section into view

```
import { motion } from "framer-motion";

const fadeUp = {
  hidden: { opacity: 0, y: 24 },
  show: { opacity: 1, y: 0, transition: { duration: 0.4 } },
};

<motion.section variants={fadeUp} initial="hidden"
  whileInView="show" viewport={{ once: true }}>
  ...
</motion.section>
```

Say aloud: “It fades and lifts into view once, only when scrolled to - cheap and non-blocking.”

The audio gate with sessionStorage

```
function useAudioChoice() {
  const [choice, setChoice] = useState<string | null>(null);
  useEffect(() => {
    setChoice(sessionStorage.getItem("audio")); // per-visit
  }, []);
  const choose = (v: string) => {
    sessionStorage.setItem("audio", v);
    setChoice(v);
  };
  return { choice, choose };
}
```

Say aloud: “sessionStorage remembers the sound choice for the visit but asks again on a fresh session.”

SEO metadata (Next.js)

```
export const metadata = {
  title: "AUKTAVE 2K26",
  description: "The official portal for the fest.",
  openGraph: { title: "AUKTAVE 2K26", images: ["/og.png"] },
};
```

Say aloud: “Structured metadata and Open Graph tags for search ranking and clean link previews.”

The hooks to drop on purpose

1. “I refactored the whole app from JavaScript to TypeScript”
2. Typed content + nested dynamic routing (data change, not code change)
3. Framer Motion vs GSAP (when each)
4. Keeping animations smooth on mid-range phones (reduced-motion, lazy)
5. PWA
6. sessionStorage vs localStorage

Each has a full answer above. Say it and pause.